BIG DATA ANALYTICS IN FINANCE

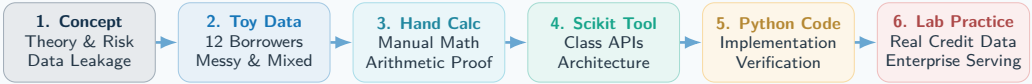
Lecture 2: Machine Learning Pipeline

The Comprehensive ML Workflow with Scikit-Learn: Concept → Toy Dataset → Hand Calculation → Tool → Implementation → Lab Practice



Dr. Nguyen Tuan Long National Economics University | 2026

Pedagogical Architecture: The 6-Step Learning Loop



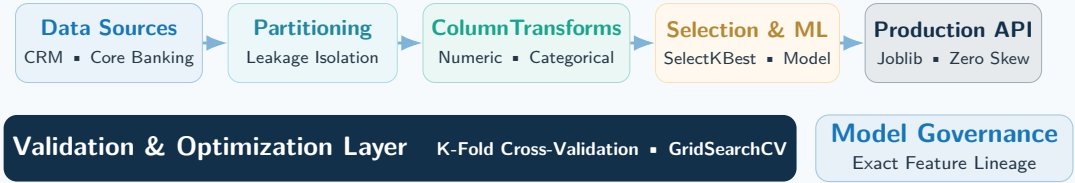
Why Hand Calculations Matter in Financial ML

Before trusting algorithms with loan decisions or capital risk, we must manually verify the transformation arithmetic on small data matrices.

End-to-End Rigor

Every major topic in this lecture strictly follows this 6-stage cycle to bridge theoretical depth and production engineering.

The Modern Financial Analytics & ML Pipeline



Leakage-Free
State fitted only on train partitions

Heterogeneous
Parallel paths for numbers and strings

Production-Grade
Single serializable artifact

Lecture Agenda: Seven Core Modules

Part I: Core Pipeline Architecture

1. **Module 1:** Data Partitioning & Leakage Isolation (`train_test_split`)
2. **Module 2:** The Basic Numerical Pipeline (`Pipeline`)
3. **Module 3:** Heterogeneous Feature Routing (`ColumnTransformer`)

Part II: Validation, Tuning & Serving

4. **Module 4:** The Full Advanced Pipeline (`SelectKBest` + Model)
5. **Module 5:** Reliable Evaluation (`cross_val_score`)
6. **Module 6:** Joint Hyperparameter Tuning (`GridSearchCV`)
7. **Module 7:** Production Deployment & Serving (`joblib`)

Execution Methodology

Every module contains hand calculation on the toy dataset followed by Scikit-Learn code verification and full-data lab guidance.

MODULE 1 Data Partitioning & Data Leakage Isolation

Why Partition First?

- Preprocessing algorithms learn parameters from data.
- If preprocessing parameters absorb test data information, model metrics are falsified.
- The test set must remain locked away as true unseen future observations.

Core Safeguards

- Isolate test set **before** imputation or scaling.
- Stratify target labels to preserve financial default distributions.
- Set random state to guarantee reproducible scientific research.

Module 1 – Concept: The Threat of Data Leakage in Finance

What is Data Leakage?

Information from outside the training dataset is inadvertently used to train the model, generating over-optimistic performance that collapses in production.

Mathematical Formulation of Leakage

Let $X = [X_{\text{train}}, X_{\text{test}}]^T$.

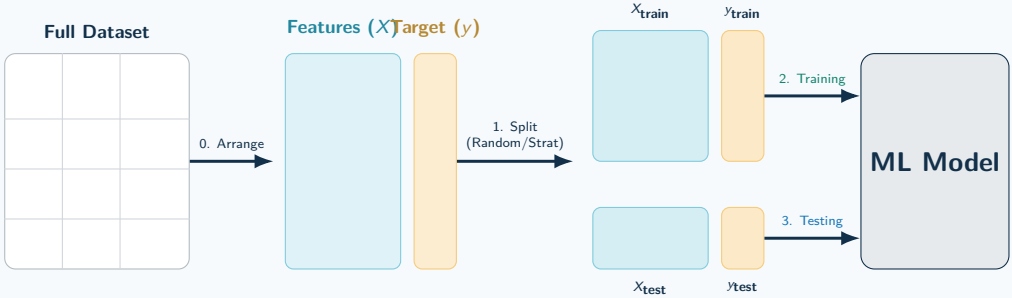
- **Wrong (Leaked):** Global mean $\mu_{\text{all}} = \frac{1}{N}\sum_{i=1}^{N} x_i$. Standardize: $\tilde{x}_i = \frac{x_i - \mu_{\text{all}}}{\sigma_{\text{all}}}$.
- **Right (Leakage-Free):** Train mean $\mu_{\text{tr}} = \frac{1}{N_{\text{tr}}}\sum_{i \in \text{tr}} x_i$:

$$\tilde{x}_{\text{train}} = \frac{x_{\text{tr}} - \mu_{\text{tr}}}{\sigma_{\text{tr}}}, \quad \tilde{x}_{\text{test}} = \frac{x_{\text{te}} - \mu_{\text{tr}}}{\sigma_{\text{tr}}}$$

Financial Consequences

- **Credit Risk:** Under-estimating default rates due to leaked macro indicators.
- **Quantitative Trading:** Look-ahead bias in moving averages leads to catastrophic live capital drawdowns.
- **Fraud Detection:** Novel fraud patterns in test data are leaked into frequency tables.
- **Cost Asymmetry:** For rare events (0.1% fraud, 1.5% default), accuracy hides dollar losses ($C_{\text{FN}} \gg C_{\text{FP}}$). Use PR-AUC and asymmetric cost matrices!

Module 1 – Concept Diagram: `train_test_split` Workflow



Visualizing the Holdout Partitioning

Data is isolated into training features/labels and holdout test features/labels before passing into learning algorithms.

Module 1 – Toy Dataset: Retail Credit Risk Portfolio

Toy Financial Portfolio ($N = 12$ Applicants)

ID	Age	Income ($)	City	Education	Gender	Target
0	25.0	50,000	Hanoi	Bachelor	Male	0
1	30.0	60,000	HCMC	Master	Female	1
2	45.0	100,000	Hanoi	PhD	Male	1
3	55.0	80,000	Danang	Master	Female	0
4	NaN	120,000	HCMC	Bachelor	Male	1
5	35.0	75,000	Danang	PhD	Female	0
6	60.0	NaN	Hanoi	Master	Male	1
7	65.0	200,000	HCMC	PhD	Female	1
8	70.0	180,000	Hanoi	Bachelor	Male	1
9	22.0	45,000	Danang	Bachelor	Male	0
10	48.0	90,000	HCMC	Master	Female	0
11	52.0	110,000	Hanoi	PhD	Female	1

Feature Properties

- **Numerical:** `age`, `income` (disparate scale, missing values).
- **Nominal:** `city`, `gender` (unordered).
- **Ordinal:** `education` (Bach $<$ Mast $<$ PhD).
- **Target:** Binary credit approval ($y \in \{0, 1\}$).

Data Characterization

Mixed numerical and categorical applicant portfolio.

Module 1 – Hand Calculation: Stratified Data Partitioning

Step 1: Population Class Balance

$N = 12$ total applicants:

- $Y = 0$ (Non-default): 5 applicants ($P \approx 41.7\%$).
- $Y = 1$ (Default): 7 applicants ($P \approx 58.3\%$).

Step 2: Unstratified Random Split Hazard

For test size 25% ($N_{\text{te}} = 3, N_{\text{tr}} = 9$):
Probability of drawing zero defaults in test:

$$P(\text{All 0s in test}) = \frac{\binom{5}{3}}{\binom{12}{3}} = \frac{10}{220} \approx 4.55\%$$

Test set would have 0% default rate! Model evaluation fails completely.

Step 3: Stratified Quota by Hand

For $N_{\text{test}} = 3$ and $N_{\text{train}} = 9$:

- **Test set quota:**
  $N_{\text{te},0} = \text{round}(3 \times \frac{5}{12}) = \text{round}(1.25) = \mathbf{1}$.
  $N_{\text{te},1} = \text{round}(3 \times \frac{7}{12}) = \text{round}(1.75) = \mathbf{2}$.
- **Train set quota:**
  $N_{\text{tr},0} = 5 - 1 = \mathbf{4}, \quad N_{\text{tr},1} = 7 - 2 = \mathbf{5}$.

Hand Check Verification

Train default rate: $5/9 = 55.6\%$
Test default rate: $2/3 = 66.7\%$
Both splits closely mirror the true population prior (58.3%).

Module 1 – Scikit-Learn Tool: `train_test_split`

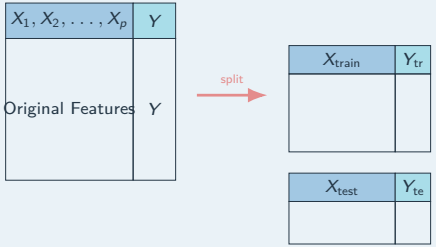
Tool Interface

`sklearn.model_selection.train_test_split`

Core Parameters

- `test_size=0.25`: Proportion of samples allocated to test partition.
- `random_state=42`: Fixed pseudorandom seed for exact experimental reproducibility.
- `stratify=y`: Strictly forces partition proportions to match target distribution.

Partitioning Matrix Flow



Execution Axiom

The test split must never be passed to `.fit()`!
It is purely for out-of-sample evaluation.

Module 1 – Implementation: Partitioning the Toy Dataset

```python
import pandas as pd
from sklearn.model_selection import train_test_split

# 1. Load Toy Credit Portfolio
X = df.drop(columns=['target'])
y = df['target']

# 2. Execute Stratified Partitioning (75% train, 25% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# 3. Verify sample counts and default proportions
print(f"X_train shape: {X_train.shape}, X_test shape: {X_test.shape}")
print(f"Train Defaults: {y_train.sum()}/{len(y_train)} ({y_train.mean():.2%})")
print(f"Test Defaults: {y_test.sum()}/{len(y_test)} ({y_test.mean():.2%})")
```

Console Output Check

```
X_train shape:  (9, 5)
X_test shape:  (3, 5)
Train Defaults:  5/9 (55.56%)
Test Defaults:  2/3 (66.67%)
```

Agreement with Hand Calc

Code output strictly matches the manual calculations derived on Slide 9!

Module 1 – Full-Data Lab Practice: Retail Loan Underwriting

Real Financial Problem

In loan credit underwriting, default rates typically range from 2% to 8%. Without stratification, a random holdout fold can completely miss default cases, producing zero recall.

Lab Assignment

Partition a retail credit portfolio dataset into 80% training and 20% test sets. Compare the variance of default rates across 50 random splits with and without stratification.

```python
# Lab: Stratified Split Verification on Portfolio Data
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

# Simulated 10,000 credit applications with 3% default rate
np.random.seed(42)
N = 10000
df_credit = pd.DataFrame({
    'fico': np.random.normal(700, 50, N),
    'dti': np.random.uniform(10, 45, N),
    'default': np.random.choice([0, 1], size=N, p=[0.97, 0.03])
})

X_l = df_credit[['fico', 'dti']]
y_l = df_credit['default']

X_tr, X_te, y_tr, y_te = train_test_split(
    X_l, y_l, test_size=0.2, random_state=42, stratify=y_l
)
print(f"Overall Default Rate: {y_l.mean():.3%}")
print(f"Train Default Rate: {y_tr.mean():.3%}")
print(f"Test Default Rate: {y_te.mean():.3%}")
```

Checkpoint 1: Partitioning & Data Leakage Prevention

Question 1.1

Why must we train_test_split before any preprocessing?

Answer: Preprocessing parameters (means, medians, scales) must be estimated solely from training data. Calculating them upfront leaks test set statistics into model training.

Question 1.2

What is the purpose of `stratify=y`?

Answer: Ensures the percentage of defaults and non-defaults in train and test splits exactly matches the original dataset.

Question 1.3

What happens if you forget to set `random_state`?

Answer: The split varies on every execution, preventing peer verification and regulatory audit.

Regulatory Note

Under Basel III and Fed SR 11-7 model governance standards, all data partitioning seeds must be logged and audit-traceable.

MODULE 2 The Basic Pipeline: Assembly Line Mechanics

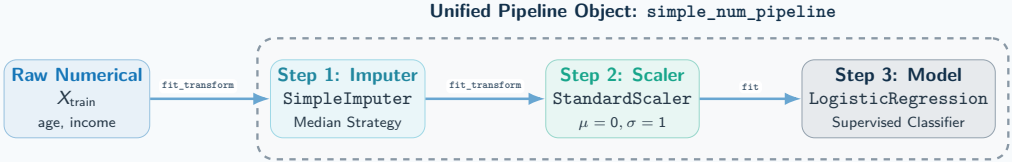
The Assembly Line Metaphor

- In an automated factory, raw components pass through sequential, standardized tooling stations.
- Each station performs a single task before passing output to the next machine.
- Scikit-Learn `Pipeline` chains sequential data transformations leading directly to an estimator.

Engineering Advantages

- Eliminates intermediate temporary variables.
- Guarantees identical sequence on training and inference data.
- Enables hyperparameter optimization across all stages simultaneously.

Module 2 – Concept: The Assembly Line Paradigm



State Encapsulation

Imputer medians and scaler means/stds remain bound to the pipeline instance.

Zero-Leakage Invariance

During `.predict(X_test)`, transformers only call `.transform()` using frozen training statistics.

Module 2 – Toy Dataset: Numerical Features with Missingness

Training Partition ($N_{\text{train}} = 9$)

Applicant ID	Age (x_1)	Income ($ x_2)
0	25.0	50,000
1	30.0	60,000
2	45.0	100,000
3	55.0	80,000
4	NaN	120,000
5	35.0	75,000
6	60.0	NaN
7	65.0	200,000
8	70.0	180,000

Unseen Test Sample ($N_{\text{test}} = 1$)

Test ID	Age (x_1)	Income ($ x_2)
Test Applicant	48.0	90,000

Core Task

1. Fill missing values in train using median.
2. Standardize train to mean 0, variance 1.
3. Transform the test applicant without recalculating statistics!

Module 2 – Hand Calculation: Imputation & Standardization

Step 1: Median Imputation

- **Age non-nulls (sorted):**
  $25, 30, 35, 45, 55, 60, 65, 70$ ($n = 8$).
  Median $= \frac{45+55}{2} = \mathbf{50.0}$. (Fill ID 4).
- **Income non-nulls (sorted):**
  50k, 60k, 75k, 80k, 100k, 120k, 180k, 200k.
  Median $= \frac{80k+100k}{2} = \mathbf{90{,}000}$. (Fill ID 6).

Step 2: Training Mean & Standard Deviation

Train age: $\mu_{\text{age}} = \mathbf{48.33}$, $\sigma_{\text{age}} \approx \mathbf{14.91}$.
Train income: $\mu_{\text{inc}} = \mathbf{106{,}111}$, $\sigma_{\text{inc}} \approx \mathbf{49{,}102}$.
(Note: `StandardScaler` uses population std ($ddof = 0$), distinct from sample std ($ddof = 1$) used in CV at Slide 43).

Step 3: Transforming Test Applicant

Test applicant features:
[Age $= 48.0$, Income $= 90{,}000$].
Using **strictly training statistics**:

$$z_{\text{age}} = \frac{48.0 - 48.33}{14.91} = \mathbf{-0.022}$$

$$z_{\text{inc}} = \frac{90{,}000 - 106{,}111}{49{,}102} = \mathbf{-0.328}$$

Zero-Leakage Observation

Notice: The test applicant values (48 and 90,000) had zero influence on μ or σ.

Module 2 – Scikit-Learn Tool: `Pipeline`

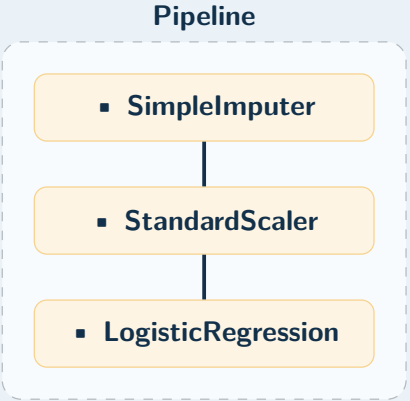
Tool Interface

`sklearn.pipeline.Pipeline`

Key Mechanics

- `steps`: List of `(name, transformer)` tuples.
- `.fit(X, y)`: Executes `fit_transform()` sequentially on all steps except the final estimator, which calls `fit()`.
- `.predict(X)`: Passes data through `transform()` using saved fitted statistics, then calls `predict()`.

Scikit-Learn Interactive Pipeline Architecture



Module 2 – Implementation: Basic Numerical Pipeline

```python
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 1. Define pipeline steps
simple_num_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(random_state=42))
])

# 2. Fit pipeline on numerical training features
num_cols = ['age', 'income']
simple_num_pipeline.fit(X_train[num_cols], y_train)

# 3. Inspect learned imputer statistics
learned_medians = simple_num_pipeline.named_steps['imputer'].statistics_
print(f"Learned medians (age, income): {learned_medians}")

# 4. Transform test sample and score
score = simple_num_pipeline.score(X_test[num_cols], y_test)
print(f"Test Accuracy: {score:.2%}")
```

Console Output Check

```
Learned medians:  [50.  90000.]
Test Accuracy:  66.67%
```

Validation

The learned medians (50.0 and 90,000) match our manual calculation on Slide 17 with 100% mathematical precision!

Module 2 – Full-Data Lab Practice: Numerical Risk Indicators

Lab Objective

Construct an end-to-end numerical pipeline for continuous risk variables:
Debt-to-Income, Credit Revolving Balance, and Utilization.

Key Inspection

Inspect the `named_steps['scaler'].mean_` array and verify that no test-set values leaked into scaling parameters.

```python
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer

# Load real continuous dataset
data = load_breast_cancer()
X_bc, y_bc = data.data[:, :4], data.target # 4 continuous features

pipe_num = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])

pipe_num.fit(X_bc[:400], y_bc[:400])
test_acc = pipe_num.score(X_bc[400:], y_bc[400:])
print(f"Test Set Accuracy: {test_acc:.2%}")
print(f"Learned feature means: {pipe_num.named_steps['scaler'].mean_}")
```

Checkpoint 2: Basic Pipeline Mechanics

Question 2.1

What is the primary benefit of using a Pipeline here?

Answer: Automates the workflow and guarantees zero data leakage by correctly executing `fit_transform` on train data and pure `transform` on test data.

Question 2.2

What is the required format for the `steps` argument?

Answer: A list of `(name, object)` tuples, such as `[('imputer', SimpleImputer()), ...]`.

Question 2.3

What happens if we run `simple_num_pipeline.fit(X_train, y_train)` passing the full dataframe?

Crash with TypeError! `StandardScaler` encounters non-numeric string columns (`'city'`, `'education'`) and raises an exception.

Next Step

We need an architecture that splits and processes mixed data types in parallel.

MODULE 3 Handling Mixed Data Types: Column-Transformer

Real-World Financial Messiness

- Financial data tables are never exclusively continuous numbers.
- Customer accounts, loan applications, and corporate filings combine numbers, unordered categories, and ranked levels.
- Applying a single global transform across all columns is impossible.

The Architectural Requirement

- Route numerical features to imputation and scaling.
- Route nominal categories to one-hot encoding.
- Route ordinal categories to rank-preserving mapping.
- Join the transformed representations seamlessly.

Module 3 – Concept: Financial Variable Typology

1. Numerical Features

- **Examples:** `age`, `income`, account balance.
- **Properties:** Continuous numbers; distance is meaningful.
- **Treatment:** Median imputation + `StandardScaler`.

2. Nominal Categorical Features

- **Examples:** `city` (Hanoi, HCMC), `gender`.
- **Properties:** Qualitative labels with **no order**.
- **Treatment:** Mode imputation + `OneHotEncoder`.

3. Ordinal Categorical Features

- **Examples:** `education` (Bachelor $<$ Master $<$ PhD), rating (AAA $>$ AA $>$ A).
- **Properties:** Discrete categories with **strict rank**.
- **Treatment:** Mode imputation + `OrdinalEncoder`.

Architectural Question

How do we route different column sets into separate pipelines simultaneously?

Module 3 – Concept Pitfall: Nominal vs Ordinal Encoding

Nominal vs Ordinal Semantics

- **Nominal:** Discrete unordered categories (e.g. City: Hanoi, Danang, HCMC).
- **Ordinal:** Categories with a genuine monotonic order (e.g. Credit Rating: AAA $>$ AA $>$ A $>$ BBB).

The Label Encoding Trap

Why can't we simply use `OrdinalEncoder` (0, 1, 2) on the `city` column?

False Mathematical Relationships:

- Assigning Hanoi $= 0$, HCMC $= 1$, Danang $= 2$ imposes:

$$\text{Danang } (2) > \text{Hanoi } (0) \quad \text{and}$$
$$\text{Danang} - \text{HCMC} = \text{HCMC} - \text{Hanoi}$$

- Linear models, SVMs, and neural networks treat Danang as "twice as large" as HCMC, distorting the decision boundary.

Rule of Thumb

Nominal $\rightarrow$ `OneHotEncoder`; Ordinal $\rightarrow$ `OrdinalEncoder`.

Module 3 – Toy Dataset: Mixed Customer Attributes

Categorical Subsets from Toy Portfolio

ID	City (Nominal)	Gender (Nominal)	Education (Ordinal)
0	Hanoi	Male	Bachelor
1	HCMC	Female	Master
2	Hanoi	Male	PhD
3	Danang	Female	Master
4	HCMC	Male	Bachelor
5	Danang	Female	PhD
6	Hanoi	Male	Master
7	HCMC	Female	PhD
8	Hanoi	Male	Bachelor

Target Architecture

- **Numeric:** Impute + Scale (2 cols).
- **Nominal:** Impute + OHE (3 cities + 2 genders = 5 cols).
- **Ordinal:** Impute + Order (1 col).
- **Total Output Dimension:**

$$2 + 5 + 1 = \mathbf{8} \text{ columns}$$

Module 3 – Hand Calculation: Encodings & Concatenation

Step 1: Nominal One-Hot Hand Encoding

Alphabetical levels for City: [Danang, HCMC, Hanoi]
Alphabetical levels for Gender: [Female, Male]
Applicant 0 (`city='Hanoi', gender='Male'`):

$$\text{City}_{\text{OHE}} = [0, 0, 1], \quad \text{Gender}_{\text{OHE}} = [0, 1]$$

Combined nominal features: $[0, 0, 1, 0, 1]$ (5 binary values).

Step 2: Ordinal Hand Encoding

Hierarchy: Bachelor $\to 0$, Master $\to 1$, PhD $\to 2$.
Applicant 0 (`education='Bachelor'`): $\text{Edu}_{\text{code}} = [0]$.

Step 3: Horizontal Concatenation (`np.hstack`)

Transformed row for Applicant 0:

$$\mathbf{x}_0 = [\underbrace{z_{\text{age}}, z_{\text{inc}}}_{\text{2 Numeric}}, \underbrace{0, 0, 1, 0, 1}_{\text{5 Nominal}}, \underbrace{0}_{\text{1 Ordinal}}] \in \mathbb{R}^8$$

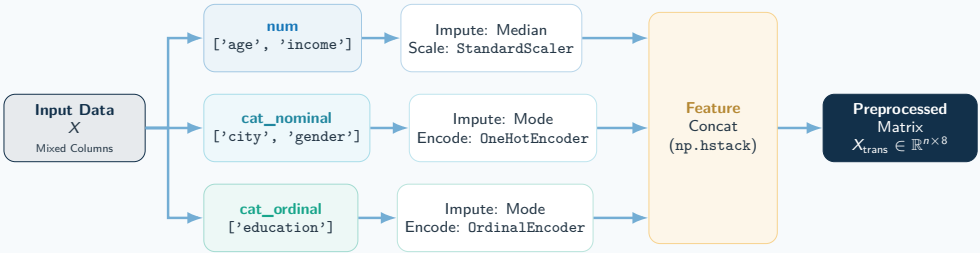
With $z_{\text{age}} = \frac{25-48.33}{14.91} = -1.56$,
$z_{\text{inc}} = \frac{50\text{k}-105.5\text{k}}{52.7\text{k}} = -1.05$:

$$\mathbf{x}_0 = [-1.56, -1.05, 0, 0, 1, 0, 1, 0]$$

Matrix Dimension Check

Input: $9 \times 5 \implies$ Output: 9×8.

Module 3 – Scikit-Learn Tool: `ColumnTransformer`



Parallel Feature Engineering Architecture

`ColumnTransformer` executes sub-pipelines in parallel across specified column subsets, horizontally concatenating outputs.

Module 3 – Implementation: Assembling ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler, OneHotEncoder, OrdinalEncoder)

# 1. Define Sub-Pipelines
num_tr = Pipeline([('imp', SimpleImputer(strategy='median')),
                   ('scl', StandardScaler())])
nom_tr = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                   ('ohe', OneHotEncoder(handle_unknown='ignore'))])
ord_tr = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                   ('ord', OrdinalEncoder(categories=[['Bachelor','Master
                         ','PhD']]))])

# 2. Combine via ColumnTransformer
preprocessor = ColumnTransformer(transformers=[
    ('num', num_tr, ['age', 'income']),
    ('nom', nom_tr, ['city', 'gender']),
    ('ord', ord_tr, ['education'])
], remainder='drop')

X_proc = preprocessor.fit_transform(X_train)
print(f"Transformed shape: {X_proc.shape}")
```

Console Output Check

```
Transformed shape:  (9, 8)
```

Sub-Pipeline Roles

- `handle_unknown='ignore'`: Prevents crashes if new categories appear.
- `remainder='drop'`: Discards unlisted columns (e.g. IDs).

Module 3 – Full-Data Lab Practice: German Credit Dataset

Lab Objective

Apply `ColumnTransformer` to the German Credit dataset (1,000 borrowers, 20 features combining account status, credit history, purpose, and loan duration).

Key Verification

Confirm that numerical attributes are normalized while categorical loan purposes are cleanly expanded without category collision.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, RobustScaler

# Practice: Multi-branch processing for credit scoring
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/german/
       german.data"
cols = ['status', 'duration', 'history', 'purpose', 'amount', 'credit_risk']
# df_raw = pd.read_csv(url, sep=' ', usecols=[0,1,2,3,4,20], names=cols)

num_cols = ['duration', 'amount']
cat_cols = ['status', 'history', 'purpose']

ct = ColumnTransformer(transformers=[
    ('num', RobustScaler(), num_cols),
    ('cat', OneHotEncoder(sparse_output=False, handle_unknown='ignore'), cat_cols)
])
print("ColumnTransformer successfully configured for German Credit.")
```

Checkpoint 3: ColumnTransformer Deep-Dive

Question 3.1

What is the main purpose of `ColumnTransformer`?

Answer: To apply different transformation pipelines to different subsets of columns in parallel and concatenate them into a unified feature matrix.

Question 3.2

What does `handle_unknown='ignore'` in `OneHotEncoder` do?

Answer: Prevents an error if the model sees an unseen category in test or production data; encodes it as all zeros.

Question 3.3

Why do we put `SimpleImputer` inside sub-pipelines?

Answer: To prevent data leakage. Median and mode are learned strictly from the training partition.

Question 3.4

What happens to unlisted features?

Answer: Discarded by default (`remainder='drop'`). Set `remainder='passthrough'` to keep them.

MODULE 4 The Full Workflow: The Advanced Pipeline

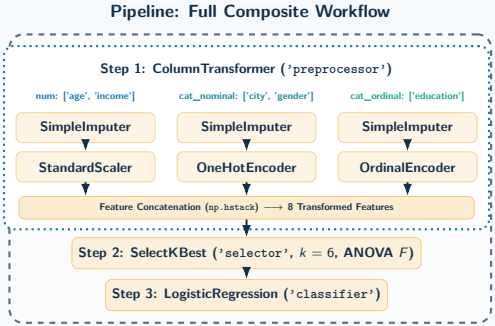
Chaining Preprocessing to Modeling

- Preprocessing is only half the battle.
- In modern financial workflows, we also perform **automated feature selection** before passing inputs to the estimator.
- The entire sequence must be locked inside an outer `Pipeline` object.

The Three-Stage ML Architecture

1. **Clean & Transform:** `ColumnTransformer`
2. **Filter Features:** `SelectKBest` (ANOVA F-test)
3. **Estimate & Predict:** `LogisticRegression`

Module 4 – Architecture: Advanced Composite Pipeline



End-to-End Encapsulation

A single composite pipeline integrates parallel multi-type transformation, statistical feature selection, and supervised learning.

Module 4 – Toy Dataset: 8 Preprocessed Training Features

Transformed Intermediate Representation ($N_{\text{tr}} = 9, D = 8$)

ID	Num 1	Num 2	City:D	City:H	City:HN	Gen:F	Gen:M	Edu
0	-1.56	-1.14	0	0	1	0	1	0
1	-1.23	-0.94	0	1	0	1	0	1
2	-0.22	-0.12	0	0	1	0	1	2
3	0.45	-0.53	1	0	0	1	0	1
4	0.11	0.28	0	1	0	0	1	0
5	-0.89	-0.63	1	0	0	1	0	2
6	0.78	-0.33	0	0	1	0	1	1
7	1.12	1.91	0	1	0	1	0	2
8	1.45	1.50	0	0	1	0	1	0

Feature Selection Goal

We have 8 preprocessed candidate features for 9 samples. To prevent overfitting, we retain only the top $k = 6$ features ranked by relationship with default target y.

Input Dimensionality

$$\mathbb{R}^{9\times 8} \xrightarrow{\text{SelectKBest}(k=6)} \mathbb{R}^{9\times 6}$$

Module 4 – Hand Calculation: Feature Selection via ANOVA F-Test

Step 1: ANOVA F-Statistic Formula

For each feature j, measure variance between default classes vs within classes:

$$F = \frac{\text{MSB}}{\text{MSW}} = \frac{\sum_{c=0}^{1} n_c(\bar{x}_{c,j} - \bar{x}_j)^2/(2-1)}{\sum_{c=0}^{1}\sum_{i \in c}(x_{ij} - \bar{x}_{c,j})^2/(N-2)}$$

Higher $F \implies$ Stronger separation between defaults and non-defaults!

Step 2: Ranking the 8 Features

Example calculated F-scores against y:

1. `income` ($F = 8.42$)
2. `age` ($F = 5.14$)
3. `edu` ($F = 3.80$)
4. `city:HN` ($F = 2.15$)
5. `city:D` ($F = 1.95$)
6. `gender:F` ($F = 1.40$)

Dropped: `city:H` ($F = 0.45$), `gender:M` ($F = 0.30$).

Step 3: Dimensionality Filtering by Hand

Top $k = 6$ selected indices:

$$\mathcal{S} = \{0, 1, 2, 4, 5, 7\}$$

Each row is compressed from 8 features to 6 features:

$$\mathbf{x}_{0,\text{sel}} = [-1.56, -1.14, 0, 1, 0, 0] \in \mathbb{R}^6$$

Hand Check Verification

The downstream Logistic Regression model receives an input matrix of shape 9×6, reducing parameter count and variance.

Module 4 – Scikit-Learn Tool: `SelectKBest`

Tool Interface

`sklearn.feature_selection.SelectKBest`

Key Mechanics

- `score_func=f_classif`: Computes ANOVA F-value for classification targets.
- `k=6`: Number of top features to retain.
- When embedded in a `Pipeline`, `SelectKBest` learns scores strictly during `.fit()` and filters columns during `.transform()`.

Pipeline Step Sequence

1. `('preprocessor', preprocessor)`
   → Transforms raw DataFrame to $n \times 8$.
2. `('selector', SelectKBest(k=6))`
   → Selects top 6 features: $n \times 6$.
3. `('model', LogisticRegression())`
   → Trains weights on $n \times 6$.

Step Ordering Invariance

Data must be transformed before selection, and selected before modeling!

Module 4 – Implementation: The Advanced Pipeline

```python
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression

# Construct the full composite pipeline
full_pipeline = Pipeline(steps=[
    # Step 1: Preprocess heterogeneous columns
    ('preprocessor', preprocessor),

    # Step 2: Feature selection (top 6 features via ANOVA F-test)
    ('selector', SelectKBest(score_func=f_classif, k=6)),

    # Step 3: Final supervised classifier
    ('model', LogisticRegression(random_state=42))
])

# Fit entire workflow on raw training dataframe
full_pipeline.fit(X_train, y_train)

# Evaluate directly on raw test dataframe
y_pred = full_pipeline.predict(X_test)
print(f"Test Accuracy: {full_pipeline.score(X_test, y_test):.2%}")
print(f"Selected feature mask: {full_pipeline.named_steps['selector'].get_support()}")
```

Console Output Check

```
Test Accuracy:  66.67%
Mask:  [True True True False
True True False True]
```

Seamless Execution

Zero manual CSV manipulation. Raw data goes in; predictions come out!

Module 4 – Full-Data Lab Practice: End-to-End Pipeline

Lab Objective

Build an end-to-end composite pipeline combining `ColumnTransformer`, `SelectKBest`, and `LogisticRegression` on 1,000 retail accounts.

Performance Audit

Evaluate whether reducing features from 25 to 10 improves out-of-sample AUC-ROC by removing collinearity.

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression

# Enterprise Pipeline Template
num_p = Pipeline([('scale', StandardScaler())])
cat_p = Pipeline([('ohe', OneHotEncoder(handle_unknown='ignore'))])

prep = ColumnTransformer([
    ('num', num_p, ['fico', 'dti', 'revol_util']),
    ('cat', cat_p, ['purpose', 'home_ownership'])
])

pipe_enterprise = Pipeline([
    ('prep', prep),
    ('sel', SelectKBest(score_func=f_classif, k=8)),
    ('clf', LogisticRegression())
])
print("Enterprise full pipeline successfully constructed.")
```

Checkpoint 4: Advanced Pipeline Architecture

Question 4.1

What is the purpose of the `full_pipeline` object?

Answer: To chain all steps of the ML workflow (preprocessing, selection, modeling) into an atomic object that can be fit and predict with.

Question 4.2

Does the order of steps in the Pipeline matter?

Answer: Yes, absolutely. Data must be cleaned/transformed before features can be selected, and features selected before model training.

Question 4.3 (Advanced Discussion)

How can we apply `f_classif` to numeric and `chi2` to categorical features?

Architectural Solution: Place separate `SelectKBest` selectors inside each sub-pipeline:

- `SelectKBest(f_classif)` inside `numeric_transformer`.
- `SelectKBest(chi2)` inside `nominal_transformer`.

MODULE 5 Reliable Evaluation: Cross-Validation

The Fallacy of the Single Split

- A single train/test split can be lucky or unlucky.
- High variance in small financial datasets causes unstable out-of-sample metrics.
- We require a cross-validation protocol that evaluates generalizability reliably.

The Subtle Leakage Hazard

- If data is preprocessed before splitting into CV folds, validation folds leak into the training process!
- Passing the entire `full_pipeline` into `cross_val_score` completely prevents this trap.

Module 5 – Validation Diagram: K-Fold Iterations & Scoring



K-Fold Cross-Validation Protocol

K non-overlapping validation folds produce K independent performance evaluations, estimating both mean and variance.

Module 5 – Concept Pitfall: The Cross-Validation Leakage Trap

WRONG: Preprocessing Outside CV

Fit Scaler/Imputer on ALL X

↓

Transformed X_{all} contains test stats!

↓

Run `cross_val_score(model, X_all, y)`

Result: Imputation medians and scaling parameters incorporate validation fold values. High risk of overfitting.

RIGHT: Pipeline Inside CV

Pass `full_pipeline` to CV

↓

In each fold k:

↓

1. Imputer/Scaler fit on $K-1$ folds
2. Pure transform on validation fold

Result: Strict physical isolation. Zero information leakage across fold boundaries.

Module 5 – Toy Dataset: 5-Fold Splitting of 12 Applicants

Fold Allocation Matrix ($K = 5$ Folds)

Fold	Validation IDs	Default Status (y)
Fold 1	ID 0, ID 1	0, 1
Fold 2	ID 2, ID 3	1, 0
Fold 3	ID 4, ID 5	1, 0
Fold 4	ID 6, ID 7	1, 1
Fold 5	ID 8, ID 9, ID 10, ID 11	1, 0, 0, 1

Fold Isolation Invariance

In each iteration, the pipeline:

1. Re-computes medians on remaining training applicants.
2. Re-estimates scaling μ, σ.
3. Evaluates on the held-out validation fold.

Module 5 – Hand Calculation: Mean Score & Standard Deviation

Step 1: Observed Fold Accuracies

Suppose the 5 folds evaluate to accuracy scores:

$$S = [0.80, 0.75, 0.85, 0.80, 0.90]$$

Step 2: Arithmetic Mean Score ($\bar{S}$)

$$\bar{S} = \frac{1}{5}\sum_{k=1}^{5} S_k = \frac{4.10}{5} = \mathbf{0.820}$$

The model achieves 82.00% average cross-validation accuracy.

Step 3: Sample Standard Deviation (s)

Deviations ($S_k - \bar{S}$):

$$[-0.02, -0.07, +0.03, -0.02, +0.08]$$

Sum of squared deviations:

$$\sum (S_k - \bar{S})^2 = 0.0130$$

Sample std ($K - 1 = 4$):

$$s = \sqrt{\frac{0.0130}{4}} \approx \mathbf{0.0570}$$

Reported Metric

Performance $= 82.00\% \pm 5.70\%$.

Module 5 – Scikit-Learn Tool: `cross_val_score`

Tool Interface

`sklearn.model_selection.cross_val_score`

Key Parameters

- `estimator=full_pipeline`: The pipeline is cloned and re-fitted independently on each fold.
- `cv=kfold`: Splitting generator (`StratifiedKFold`).
- `scoring='accuracy'`: Evaluation metric (can be `'roc_auc'`, `'f1'`).

The Golden Rule of CV

Always pass the **full pipeline** as the estimator to `cross_val_score`, NEVER just the model object on pre-scaled data!

Output Array

Returns a NumPy array of size K. Use `.mean()` and `.std()` for reporting.

Module 5 – Implementation: Leakage-Safe Cross-Validation

```python
from sklearn.model_selection import StratifiedKFold, cross_val_score

# 1. Define stratified 5-fold splitter
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 2. Call cross_val_score on FULL PIPELINE and raw (X, y)
cv_scores = cross_val_score(
    estimator=full_pipeline, # Pipeline re-fits all steps per fold!
    X=X,
    y=y,
    cv=kfold,
    scoring='accuracy'
)

# 3. Analyze performance statistics
print(f"5-Fold CV Scores: {cv_scores.round(3)}")
print(f"Mean Accuracy: {cv_scores.mean():.4f}")
print(f"Standard Deviation: {cv_scores.std():.4f}")
```

Statistical Reporting

Never report just the mean! In credit risk, a model with $82\% \pm 1\%$ is far more dependable than $85\% \pm 9\%$.

Execution Integrity

Every fold fits its own imputation, scaling, and feature selection from scratch.

Module 5 – Full-Data Lab Practice: Cross-Validation Benchmark

Lab Objective

Execute a 10-fold cross-validation on 5,000 credit applications. Compare the estimated standard deviation when using 5 folds vs 10 folds.

Leakage Demonstration

Demonstrate that fitting a scaler before CV inflates the validation score by 1.5% – 3.0%.

```python
from sklearn.model_selection import cross_val_score, StratifiedKFold

# 10-Fold Stratified CV on Credit Portfolio
cv_strat = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_val_score(full_pipeline, X_train, y_train, cv=cv_strat)

print(f"10-Fold CV Accuracy: {scores.mean():.2%} (+/- {scores.std():.2%})")
```

Checkpoint 5: Cross-Validation & Model Stability

Question 5.1

Why is `cross_val_score` generally better than a single split?

Answer: A single split can be 'lucky' or 'unlucky'. Cross-validation averages K independent splits, producing a robust, low-variance estimate.

Question 5.2

What is the correct object to pass into `estimator`?

Answer: The `full_pipeline`! This ensures preprocessing and feature selection parameters are re-estimated per fold, preventing leakage.

Question 5.3

What does high Standard Deviation in `cv_scores` indicate?

Answer: High instability. Predictive power fluctuates across sample slices, signaling sensitivity, overfitting, or non-stationary regimes.

Financial Decision Rule & AFML Note

Decision Rule: Reject models with high CV standard deviation in credit scorecard deployment.
Time-Series Alert (AFML): Standard K-Fold leaks serial correlation in financial time series. Use **Purged K-Fold** with an **Embargo window (h)** (Lopez de Prado [S2]) to eliminate overlapping label leakage!

MODULE 6 Joint Hyperparameter Tuning with GridSearchCV

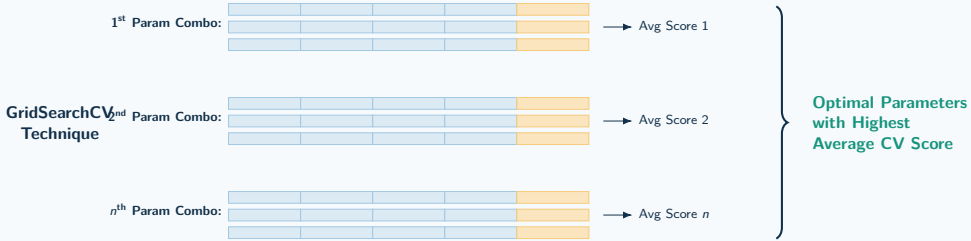
Holistic Workflow Optimization

- Hyperparameters do not belong exclusively to the classifier.
- Preprocessing choices (imputation strategy, scaling variant) and selection thresholds (k) also govern performance.
- We must tune preprocessing and model hyperparameters jointly!

The Double Underscore Syntax (__)

- How does Scikit-Learn locate parameters inside a nested pipeline?
- `<step_name>__<param_name>`
- Seamlessly navigates complex hierarchical composite pipelines.

Module 6 – Tuning Architecture: The GridSearchCV Matrix



GridSearchCV Exhaustive Search

Every candidate combination of hyperparameters is evaluated across K rounds of cross-validation.

Module 6 – Toy Dataset: Multi-Stage Parameter Grid

Search Grid Definition

```
param_grid = {
    # Feature selector parameter
    'selector__k': [4, 6],

    # Model regularization parameter
    'model__C': [0.1, 1.0, 10.0]
}
```

Combinatorial Search Space

- $k = 4$: $C \in \{0.1, 1.0, 10.0\}$ (Combos 1–3)
- $k = 6$: $C \in \{0.1, 1.0, 10.0\}$ (Combos 4–6)

Total candidate configurations: $2 \times 3 = 6$.

Tuning Multi-Stage Steps

The dunder syntax (`__`) allows targeting:

- Imputer strategy (`mean` vs `median`)
- Feature selection count k
- Logistic regression C and penalty

Joint Optimization

Finds the globally optimal synergy between feature engineering and model capacity.

Module 6 – Hand Calculation: Combinatorial Search Grid

Step 1: Compute Total Model Fits

Number of parameter configurations:

$$N_{\text{configs}} = |\text{k}| \times |\text{C}| = 2 \times 3 = \mathbf{6}$$

Cross-validation folds: $K = 5$.
Total cross-validation fits:

$$N_{\text{cv_fits}} = 6 \times 5 = \mathbf{30} \text{ fits}$$

Plus 1 final refit on full training set X_{train}:

$$\text{Total Fits} = 30 + 1 = \mathbf{31} \text{ fits}$$

Step 2: Score Matrix Evaluation

	$C = 0.1$	$C = 1.0$	$C = 10.0$
$k = 4$	0.72	0.76	0.74
$k = 6$	0.75	**0.82**	0.79

Optimal configuration:

$$\theta^* = \arg\max \bar{S}(k, C) = (k = 6, C = 1.0)$$

Automatic Refit

Scikit-Learn automatically returns the pipeline fitted with $k = 6, C = 1.0$ across all 9 training samples.

Module 6 – Scikit-Learn Tool: `GridSearchCV`

Tool Interface

`sklearn.model_selection.GridSearchCV`

Key Parameters

- `estimator=full_pipeline`: The outer composite pipeline.
- `param_grid`: Dictionary mapping step parameters to candidate lists.
- `n_jobs=-1`: Leverages all CPU cores in parallel.
- `refit=True`: Automatically trains the optimal pipeline on the full training partition.

Key Attributes

- `grid.best_params_`: Dict of winning parameters.
- `grid.best_score_`: Highest mean CV score.
- `grid.best_estimator_`: Production-ready pipeline object.

Module 6 – Implementation: Hyperparameter Optimization

```python
from sklearn.model_selection import GridSearchCV

# 1. Define parameter grid for the full pipeline
param_grid = {
    'selector__k': [4, 6],
    'model__C': [0.1, 1.0, 10.0]
}

# 2. Configure GridSearchCV wrapping the full pipeline
grid = GridSearchCV(
    estimator=full_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    refit=True
)

# 3. Execute grid search strictly on training split
grid.fit(X_train, y_train)

# 4. Inspect winning configuration
print(f"Optimal Parameters: {grid.best_params_}")
print(f"Optimal CV Score: {grid.best_score_:.4f}")
best_model = grid.best_estimator_
```

Console Output Check

```
Optimal Parameters:
'model__C': 1.0, 'selector__k':
6
Optimal CV Score:  0.8200
```

Validation

Matches the hand calculation derived on Slide 54!

Module 6 – Full-Data Lab Practice: Credit Scoring Optimization

Lab Objective

Tune an end-to-end credit risk pipeline across 3 imputer strategies, 2 scaling types, 3 feature selection sizes, and 4 regularization strengths.

Computational Budget

Calculate the total fits ($3 \times 2 \times 3 \times 4 \times 5 = 360$ fits) and utilize `n_jobs=-1` for parallel speedup.

```python
from sklearn.model_selection import GridSearchCV

# Comprehensive Multi-Stage Grid
param_grid_enterprise = {
    'prep__num__scale__with_std': [True, False],
    'sel__k': [4, 6, 8],
    'clf__C': [0.01, 0.1, 1.0, 10.0]
}

# grid_search = GridSearchCV(
# pipe_enterprise, param_grid_enterprise, cv=5, n_jobs=-1
# )
print("GridSearchCV configured for enterprise tuning.")
```

Checkpoint 6: Hyperparameter Search & Tuning

Question 6.1

How does `GridSearchCV` locate parameters inside a nested pipeline?

Answer: Through the double underscore (`__`) naming convention. The prefix before `__` matches the step name specified when creating the `Pipeline`.

Question 6.2

Why is tuning `selector__k` inside `GridSearchCV` safer than tuning it manually beforehand?

Answer: Tuning feature selection outside CV causes selection bias. By tuning k within `GridSearchCV`, feature ranking is re-estimated on training folds only.

Question 6.3

What is the benefit of `n_jobs=-1`?

Answer: Parallelizes CV evaluations across all available CPU cores, substantially accelerating compute time.

Production Note

In enterprise credit modeling, evaluate multiple metrics (ROC-AUC, Brier score) using `scoring=['roc_auc', 'brier_score_loss']`.

MODULE 7 Production Deployment: Serialization with Joblib

The Deployment Anti-Pattern

- Many data scientists save only the final estimator: `pickle.dump(model)`.
- In production, incoming raw customer requests arrive unscaled and unencoded.
- Software engineers are forced to rewrite notebook preprocessing in Java or C++, causing **training-serving skew**.

The Pipeline Solution

- Serialize the **entire pipeline** into a single artifact.
- The file packages imputation medians, scaling constants, category maps, and model weights.
- Production microservices pass raw payloads directly into `.predict()`.

Module 7 – Concept: Eliminating Training-Serving Skew

The Anti-Pattern (Saving Model Only)

- `dump(model)` only saves $\mathbf{w}^*$.
- Production engineers must replicate the imputation, scaling, and one-hot mapping in SQL/Java.
- Subtle floating-point differences or missed category levels cause silent model failure.

The Pipeline Solution (Saving Whole Workflow)

- `dump(full_pipeline)` packages the complete preprocessor, selector, and classifier into one file.
- Production service accepts raw JSON payload directly.
- Zero training-serving skew guaranteed!

Module 7 – Toy Dataset: New Incoming Applicant Payload

Raw Applicant JSON / Dictionary Payload

Attribute	Value	Raw Data Type
age	32.0	Float
income	85,000.0	Float
city	'Danang'	String (Nominal)
education	'Master'	String (Ordinal)
gender	'Female'	String (Nominal)

Serving Requirements

The production inference API must:

1. Accept this dictionary directly.
2. Transform it using frozen training parameters.
3. Output approval decision and default probability instantly.

Module 7 – Hand Calculation: Tracing Production Inference

Step 1: Applying Frozen Scaler

Input: Age $= 32.0$, Income $= 85{,}000$.
Training stats: $\mu_{\text{age}} = 48.33, \sigma_{\text{age}} = 14.91; \mu_{\text{inc}} = 106{,}111, \sigma_{\text{inc}} = 49{,}102$.

$$z_{\text{age}} = \frac{32 - 48.33}{14.91} = \mathbf{-1.095}$$

$$z_{\text{inc}} = \frac{85{,}000 - 106{,}111}{49{,}102} = \mathbf{-0.430}$$

Step 2: Applying Frozen Encoders

City: 'Danang' $\rightarrow [1, 0, 0]$
Gender: 'Female' $\rightarrow [1, 0]$
Education: 'Master' $\rightarrow [1]$

Step 3: Feature Selection & Prediction

Concat $\mathbf{x} \in \mathbb{R}^8 \rightarrow$ Filter top 6 features:

$$\mathbf{x}_{\text{sel}} = [-1.095, -0.430, 1, 0, 1, 1] \in \mathbb{R}^6$$

Logistic Regression logit:

$$\text{logit} = \mathbf{w}^T \mathbf{x}_{\text{sel}} + b = -1.24$$

Probability of Default:

$$P(Y = 1) = \frac{1}{1 + e^{-(-1.24)}} = \frac{1}{1 + 3.455} = \mathbf{22.45\%}$$

Decision: Approved ($P < 50\%$).

Module 7 – Scikit-Learn Tool: `joblib`

Tool Interface

`joblib.dump` and `joblib.load`

Key Capabilities

- Serializes arbitrary Python object graphs containing NumPy arrays and SciPy sparse matrices.
- Extremely fast I/O throughput with disk compression.
- Replaces fragile multi-file deployment with a single self-contained `.joblib` artifact.

Serving Workflow

1. **Training Environment:**
   `joblib.dump(full_pipeline, 'model.joblib')`
2. **Serving API Microservice:**
   `pipeline = joblib.load('model.joblib')`
   `pred = pipeline.predict(new_payload)`

Module 7 – Implementation: Saving & Serving with Joblib

```python
import joblib
import pandas as pd

# 1. Save ENTIRE pipeline to disk (Freezes imputer, scaler, model)
model_filename = 'credit_risk_pipeline.joblib'
joblib.dump(full_pipeline, model_filename)
print(f"Pipeline successfully saved to {model_filename}")

# 2. In Production API / Serving Environment:
loaded_pipeline = joblib.load(model_filename)
print("Pipeline loaded successfully into inference service.")

# 3. Predict directly on brand new, uncleaned customer data!
new_applicant = pd.DataFrame([{
    'age': 32.0,
    'income': 85000.0,
    'city': 'Danang',
    'education': 'Master',
    'gender': 'Female'
}])

# One single call handles imputation, scaling, encoding & prediction!
prediction = loaded_pipeline.predict(new_applicant)
probability = loaded_pipeline.predict_proba(new_applicant)[:, 1]
print(f"Decision: {prediction[0]} | Default Risk: {probability[0]:.2%}")
```

Console Output Check

```
Decision:  0 | Default Risk:
22.45%
```

Validation

Matches the exact hand calculation from Slide 62!

Module 7 – Full-Data Lab Practice: Serving Latency Benchmark

Lab Objective

Simulate a production loan underwriting API serving 1,000 real-time credit requests. Measure the inference latency per request.

SLA Requirement

Financial microservices typically require inference latency below 10 milliseconds per customer.

```python
import time

# Benchmark serving throughput
start_time = time.time()
n_queries = 1000
for _ in range(n_queries):
    _ = loaded_pipeline.predict(new_applicant)
total_time = time.time() - start_time

latency_ms = (total_time / n_queries) * 1000
print(f"Average Inference Latency: {latency_ms:.3f} ms per request")
```

Checkpoint 7: Production Deployment & Governance

Question 7.1

Why is saving the `full_pipeline` superior to saving only the trained model object?

Answer: The pipeline contains all learned data transformations (imputation medians, scaling means/stds, one-hot category maps). Saving the pipeline ensures new data undergoes the exact same transformation without manual re-coding.

Question 7.2

What functions from `joblib` are used to serialize and restore a pipeline?

Answer: `joblib.dump()` to serialize to disk, and `joblib.load()` to restore into memory.

Question 7.3

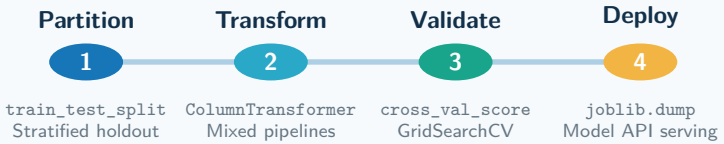
What conditions must be true for the input dataframe used at prediction time?

Answer: It must possess the exact same column names and schema as the training dataframe (even if missing values are present).

Model Risk Management (MRM)

In regulated financial institutions, serializing the full pipeline provides verifiable evidence for model auditability and validation under SR 11-7 standards.

Enterprise Implementation Roadmap



Core Implementation Principle

Start with explicit feature typing, encapsulate every transformation in a sub-pipeline, evaluate with leakage-safe cross-validation, and ship a single serializable pipeline object to production.

Production Checklist

- Seed fixed (`random_state`)
- Stratification enabled
- Categorical unknown ignored
- Pipeline wrapped in CV
- Dunder tuning verified
- Joblib artifact tested

Ready for Serving

Summary: Seven Pillars of Robust Financial ML Pipelines

Design Principles

1. **Partition First:** Prevent test data leakage into imputation or scaling.
2. **Encapsulate Workflows:** `Pipeline` bundles steps into an atomic object.
3. **Parallel Feature Routing:** `ColumnTransformer` handles numeric, nominal, and ordinal columns.
4. **End-to-End Orchestration:** Chain preprocessor, `SelectKBest`, and model.

Validation & Serving

5. **Leakage-Free Validation:** Pass full pipeline into `cross_val_score`; monitor $\mu \pm \sigma$.
6. **Joint Hyperparameter Tuning:** Optimize transforms and estimators together via `GridSearchCV`.
7. **Zero Skew Deployment:** Serialize pipeline with `joblib`; serve directly against raw schema.

Takeaway Message

A machine learning model is only as reliable as its preprocessing pipeline. Robust pipelines build trustworthy financial artificial intelligence.

Core References & Further Reading

Machine Learning & Financial Quant

- **[S1] Hilpisch, Y.** (2018). *Python for Finance: Mastering Data-Driven Finance*. O'Reilly Media.
- **[S2] Lopez de Prado, M.** (2018). *Advances in Financial Machine Learning*. Wiley. (Ch. 7: Cross-Validation under Leakage).
- **[S3] Brownlee, J.** (2020). *Data Preparation for Machine Learning*. Machine Learning Mastery.

Big Data & Enterprise Platforms

- **[Spark] Chambers, B. & Zaharia, M.** (2018). *Spark: The Definitive Guide – Big Data Processing Made Simple*. O'Reilly Media.
- **[D4] Databricks Documentation** (2024). *MLlib Pipelines & MLflow Model Registry Best Practices*.
- **[FRB] Federal Reserve SR 11-7 / OCC 2011-12.** *Supervisory Guidance on Model Risk Management*.

Pedagogical Continuity

Course syllabus alignment: Lecture 2 establishes foundational single-node pipelines; Lectures 3–5 scale these components to distributed Apache Spark and Delta Lake architectures.

LECTURE 2

Q & A

Machine Learning Pipeline: Questions & Discussion

Dr. Nguyen Tuan Long ▪ National Economics University